

AgentBench-FR : Évaluation comparative des frameworks d'agents IA

Equipe de recherche - Projet_ia

22 mai 2026

Résumé

Cet article présente AgentBench-FR, une suite d'évaluation comparative conçue pour mesurer et comparer les performances de frameworks d'agents IA (LangChain, CrewAI, AutoGen et LlamaIndex) dans un environnement local et reproductible. Nous exposons la problématique de l'évaluation inter-frameworks, détaillons une méthodologie garantissant l'équité expérimentale (mêmes modèles, mêmes matériels, mêmes tâches) et proposons un protocole d'exécution pour 120 tâches réparties en cinq catégories. Les métriques retenues couvrent la qualité fonctionnelle (taux de complétion), la performance (latence, nombre d'appels LLM), la consommation de ressources (RAM, GPU) et la consommation de tokens. Les résultats empiriques obtenus sont présentés de manière factuelle et accompagnés d'une discussion interprétative mettant en relation les différences observées avec les choix architecturaux des frameworks étudiés.

Abstract (English) : This report introduces AgentBench-FR, a reproducible local benchmark for comparing agent frameworks (LangChain, CrewAI, AutoGen, LlamaIndex). We describe the evaluation methodology, experimental protocol for 120 tasks across five categories, and the set of metrics used to assess functional quality, performance, resource usage and token consumption. Empirical findings are presented and discussed with regard to architectural differences between frameworks.

Table des matières

Introduction

La montée en puissance des modèles de langage a transformé le rôle des applications d’IA : il ne s’agit plus seulement de générer du texte, mais d’enchaîner des actions, de consulter des outils externes, de conserver un état de travail et de coordonner plusieurs sous-agents. Cette évolution a fait émerger un écosystème foisonnant de frameworks d’agents, chacun proposant ses abstractions, son style d’orchestration et ses conventions de programmation. La conséquence directe est un problème méthodologique : les comparaisons informelles deviennent vite trompeuses, car elles mélangent souvent les effets du modèle, du matériel, du backend API et du cadre d’orchestration.

AgentBench-FR répond à ce problème en imposant un environnement local unique, un même modèle, des tâches identiques et des règles d’exécution strictes. L’objectif n’est pas de proclamer un vainqueur absolu, mais de rendre mesurable ce qui différencie réellement les frameworks. Cette démarche est utile pour la recherche, mais aussi pour les équipes de développement qui doivent choisir une pile technique avant d’engager du temps d’intégration. Trois questions structurent l’étude : quelles différences de performance observe-t-on entre les frameworks, quels mécanismes architecturaux expliquent ces écarts, et quelles recommandations pratiques peut-on formuler selon le type d’application visé ?

Le reste du document suit une logique progressive. Le premier chapitre situe la contribution dans la littérature sur l’évaluation des agents et des benchmarks de raisonnement. Le deuxième chapitre décrit les frameworks étudiés, afin de relier leurs architectures à leurs usages naturels. Le troisième chapitre explicite la méthodologie, tandis que le quatrième détaille le protocole expérimental exact. Les cinquième et sixième chapitres présentent, respectivement, les résultats factuels puis leur interprétation. Les annexes enfin assurent la reproductibilité en documentant les tâches, les paramètres, les sources et les résultats détaillés.

1 État de l’art

La littérature récente sur les agents IA s’organise autour de plusieurs axes complémentaires : la définition du comportement agentique, l’évaluation des capacités de raisonnement et d’action, l’utilisation d’outils externes, et la robustesse des systèmes face à des tâches ouvertes. Dans ce contexte, ReAct [?] a proposé un schéma de raisonnement-action devenu central dans la conception des agents modernes. L’idée est simple mais puissante : un modèle doit pouvoir alterner entre raisonnement explicite et appel d’outils, afin de résoudre des problèmes nécessitant à la fois planification et interaction.

1.1 Les paradigmes fondamentaux

Un agent se distingue d’une simple chaîne de traitement par sa capacité à modifier sa trajectoire au fil de l’exécution. Il peut conserver un état, sélectionner un outil, reformuler une requête, ou encore déléguer une sous-tâche. Cette logique se retrouve sous des formes différentes dans les frameworks évalués : orchestration en chaîne dans LangChain, équipe de rôles dans CrewAI, conversation multi-agents dans AutoGen, et récupération documentaire augmentée dans LlamaIndex. Ces paradigmes ne sont pas interchangeables ; ils orientent la manière dont les erreurs apparaissent, dont les tâches sont découpées et dont le système réagit aux ambiguïtés.

1.2 Benchmarks de référence

Les benchmarks existants, comme AgentBench [?], WebArena [?], ToolBench [?] et GAIA [?], ont chacun apporté une contribution importante. AgentBench a popularisé l’idée d’une évaluation large des agents sur différents environnements. WebArena a montré l’intérêt d’un environnement web réaliste pour tester l’autonomie. ToolBench a renforcé la question du bon usage des outils, tandis que GAIA a mis l’accent sur la généralisation de l’assistant dans des tâches hétérogènes.

Néanmoins, ces travaux n'ont pas pour objectif principal de comparer plusieurs frameworks d'orchestration sur un backend identique ; ils évaluent le comportement global de l'agent, mais pas l'influence précise de la couche d'orchestration.

1.3 Vide identifié

Le vide scientifique que comble AgentBench-FR est donc précis : comparer plusieurs frameworks d'agents, dans des conditions contrôlées, avec un même modèle local et une même suite de tâches contextualisées. Cette contrainte est essentielle car elle élimine plusieurs biais habituels des benchmarks web : variation de version du modèle, dépendance à un service distant, impossibilité d'inspecter la pile complète d'exécution, et difficulté à reproduire un résultat plusieurs semaines plus tard. Notre approche permet ainsi de focaliser l'analyse sur ce qui relève réellement du framework.

2 Frameworks étudiés

Cette section présente les quatre frameworks évalués et explicite le lien entre leur architecture et les types de tâches qu'ils favorisent. Les descriptions s'appuient sur la documentation officielle des projets [?, ?, ?, ?, ?].

2.1 LangChain / LangGraph

LangChain se distingue par une approche en briques : chaînes de prompts, fonctions outillées, mémoire conversationnelle, agents et intégrations vers des systèmes externes. LangGraph ajoute une dimension de graphe à cette logique en permettant une orchestration explicite des états et des transitions. Sur le plan pratique, ce paradigme est particulièrement intéressant pour les pipelines modulaires et les applications qui doivent rester lisibles pour une équipe de développement. Son principal coût est la complexité de configuration ; plus l'architecture devient expressive, plus la surface de réglages s'élargit.

Dans le contexte d'AgentBench-FR, LangChain est attendu comme un framework équilibré : suffisamment généraliste pour couvrir plusieurs catégories de tâches, mais sans spécialisation excessive dans les scénarios multi-agent ou documentaires purs. Cette position intermédiaire en fait un bon point de référence pour comparer l'effet des autres paradigmes.

2.2 CrewAI

CrewAI repose sur une idée simple : attribuer des rôles clairs à plusieurs agents et organiser leur coopération autour d'objectifs partagés. Cette logique de « crew » se prête naturellement aux tâches où la répartition du travail apporte un bénéfice conceptuel, par exemple la recherche d'information combinée à une synthèse finale, ou la résolution d'un problème nécessitant plusieurs points de vue. L'intérêt principal du framework réside dans sa lisibilité : les rôles rendent le workflow facile à expliquer et à auditer.

En contrepartie, la spécialisation peut devenir une limite lorsque la tâche est très séquentielle et qu'une simple chaîne suffirait. Dans ce cas, l'abstraction supplémentaire peut introduire un peu de surcharge sans apporter de gain fonctionnel immédiat. Ce type d'écart est précisément ce qu'un benchmark comparatif doit mesurer.

2.3 AutoGen

AutoGen adopte un paradigme conversationnel : les agents dialoguent, se répondent, débattent et affinent leur stratégie au fil des tours. Ce design est particulièrement utile lorsque la tâche demande de l'itération, du contre-argument ou une coordination non triviale entre plusieurs

sous-objectifs. Il peut également favoriser des comportements émergents intéressants, notamment lorsque plusieurs agents ont des responsabilités distinctes.

Le revers de cette richesse est la latence. Chaque tour supplémentaire coûte du temps, augmente la consommation de tokens et multiplie les occasions d’erreur de format ou de boucle de discussion. Sur des tâches simples, cet overhead peut devenir inutile ; sur des tâches complexes, il constitue parfois le prix d’une résolution plus robuste.

2.4 LlamaIndex

LlamaIndex est conçu autour de l’accès à l’information et de la récupération documentaire. Le framework excelle lorsqu’il faut indexer, requêter, filtrer et synthétiser un corpus. Il est donc très pertinent pour les tâches de type RAG, pour les systèmes de question-réponse documentaire ou pour les assistants qui doivent s’appuyer sur une base de connaissances locale.

Cette spécialisation le rend particulièrement performant sur les tâches documentaires, mais potentiellement moins avantageux sur les scénarios qui ne mobilisent pas de corpus. LlamaIndex peut alors sembler moins spectaculaire, non pas parce qu’il serait moins capable, mais parce que le benchmark ne sollicite pas son point fort naturel. C’est précisément pour éviter ce biais que la suite de tâches d’AgentBench-FR combine plusieurs catégories.

2.5 Synthèse comparative

Framework	Paradigme	Point fort naturel	Limite principale	Usage recommandé
LangChain	Chaînes et graphes	Modularité et éco-système	Configuration plus lourde	Pipelines hybrides
CrewAI	Rôles et équipes	Lisibilité du multi-agent	Moins direct pour les flux simples	Coordination d’agents
AutoGen	Conversation multi-agents	Collaboration émergente	Latence et tours supplémentaires	Débats et coopération
LlamaIndex	RAG et requêtage	Tâches documentaires	Moins utile hors corpus	Recherche sur base de connaissances

TABLE 1 – Synthèse conceptuelle des quatre frameworks étudiés.

3 Méthodologie

La méthodologie adoptée vise à isoler la variable « framework » en contrôlant l’ensemble des autres facteurs expérimentaux. Le principe est classique en expérimentation comparative : si l’on veut comprendre l’effet d’une seule variable, il faut maintenir constantes les autres. Ici, cela signifie que le modèle, la machine, les tâches, les paramètres de génération et la manière de juger les sorties restent alignés autant que possible.

3.1 Principes méthodologiques retenus

Le premier principe est la neutralité du modèle. Tous les frameworks s’appuient sur un backend local unique via Ollama, de manière à éviter les biais liés à des API différentes ou à des politiques tarifaires variables. Le deuxième principe est l’homogénéité matérielle : un même poste de travail, un même GPU et un même système d’exploitation servent à toutes les exécutions. Le troisième principe est la diversité contrôlée des tâches : la suite couvre plusieurs familles cognitives afin de ne pas favoriser un framework en particulier.

Catégorie	N1	N2	N3	N4	N5	Total
Recherche d’information	5	5	5	5	5	25
Raisonnement séquentiel	5	5	5	5	5	25
Récupération documentaire	5	5	5	5	5	25
Collaboration multi-agent	5	5	5	5	5	25
Gestion d’erreurs et ambiguïtés	5	5	5	5	5	25

TABLE 2 – Répartition conceptuelle des 120 tâches par catégorie et niveau de difficulté.

3.2 Métriques retenues

Les métriques principales sont choisies pour couvrir trois dimensions. La première est la réussite fonctionnelle, mesurée par le taux de complétion. La seconde est la performance temporelle, via la latence médiane et moyenne. La troisième est le coût opérationnel, via les tokens et les ressources matérielles. Les formules utilisées sont simples et explicites :

$$TCR = \frac{N_{\text{succès}}}{N_{\text{total}}} \quad (1)$$

$$LMT = \text{médiane}(t_1, t_2, \dots, t_n) \quad (2)$$

$$CTT = \sum_{i=1}^n (p_i + c_i) \quad (3)$$

Le taux de complétion (TCR) reflète la capacité du framework à terminer correctement les tâches. La latence médiane (LMT) est retenue plutôt que la moyenne seule, car elle est moins sensible aux valeurs extrêmes. La consommation totale de tokens (CTT) agrège les tokens d’entrée et de sortie, ce qui permet de comparer l’efficacité conversationnelle. Les métriques secondaires complètent l’analyse sans la dominer : delta mémoire RAM, consommation GPU et nombre d’appels LLM.

3.3 Système d’évaluation de la qualité

Pour les tâches objectives, une vérification automatique est appliquée. Pour les tâches à sortie structurée, un contrôle de schéma valide la conformité du résultat. Pour les tâches ouvertes, un protocole LLM-as-judge est utilisé avec une grille fixe, puis un sous-ensemble est vérifié manuellement afin de contrôler la cohérence du jugement. Cette combinaison limite l’arbitraire tout en gardant une évaluation praticable sur 120 tâches.

3.4 Taxonomie des erreurs

La taxonomie adoptée distingue cinq modes d’échec : `TIMEOUT`, `PARSE_ERROR`, `TOOL_ERROR`, `LOOP` et `CONTEXT_OVERFLOW`. `TIMEOUT` signale qu’une tâche n’a pas terminé dans le temps imparti. `PARSE_ERROR` correspond à une réponse qui ne respecte pas le format attendu. `TOOL_ERROR` indique un échec d’appel d’outil, souvent lié à une incohérence d’argument ou à un retour imprévu. `LOOP` décrit un comportement cyclique où l’agent répète des étapes sans converger. `CONTEXT_OVERFLOW` enfin désigne un dépassement de fenêtre de contexte ou une saturation de mémoire conversationnelle.

4 Protocole expérimental

Le protocole détaille l’environnement logiciel et matériel, la configuration harmonisée des frameworks, la procédure d’exécution et le stockage des artefacts. Il a été conçu pour que tout

lecteur puisse comprendre non seulement ce qui a été mesuré, mais aussi dans quelles conditions exactes la mesure a été produite.

4.1 Environnement matériel et logiciel

Les expériences ont été menées sur une station de travail locale équipée d’un GPU NVIDIA, avec Python 3.11 et Ollama installé en local. La modélisation s’appuie sur un seul backend de génération afin de neutraliser les effets de variation inter-modèles. Les frameworks ont été exécutés dans un environnement virtuel Python dédié, ce qui garantit une isolation correcte des dépendances et une plus grande stabilité du protocole.

4.2 Configuration harmonisée des frameworks

Les paramètres de génération ont été harmonisés pour tous les frameworks : `temperature=0` pour éliminer le bruit aléatoire, `max_tokens` fixé dans une plage cohérente, et `timeout` identique pour chaque tâche. Les sorties ont été contraintes à un format exploitable par l’analyse automatique, afin de ne pas pénaliser un framework en raison d’un simple problème de parsing. Le tableau ci-dessous résume les principaux réglages.

Paramètre	Valeur	Justification
Backend	Ollama local	Indépendance vis-à-vis des API propriétaires
Température	0	Réduction de la variance de génération
Timeout	Uniforme	Comparabilité des temps d’exécution
Format de sortie	Structuré	Évaluation automatique fiable
Répétitions	1 run de référence, avec logs détaillés	Reproductibilité et traçabilité

TABLE 3 – Paramètres harmonisés du protocole expérimental.

4.3 Procédure d’exécution

Les tâches sont exécutées en run à froid afin de limiter l’influence du cache et des états résiduels. Entre deux tâches, le contexte du backend local est réinitialisé. Chaque exécution produit un log détaillé comprenant la latence, les métriques de ressources, les tokens et les éventuelles erreurs. Les résultats sont ensuite consolidés dans des fichiers CSV et JSON, ce qui simplifie l’audit a posteriori.

4.4 Collecte et stockage des données

Les résultats bruts sont enregistrés dans `logs/`, les agrégats dans `results/summary_by_framework.csv` et `results/summary_by_category.csv`, et les analyses globales dans `results/analysis.json`. Cette séparation des niveaux de données est importante : les logs détaillés servent à l’audit, les CSV facilitent les comparaisons, et le JSON fournit un résumé pour la publication et l’exploration rapide.

4.5 Comparaison avec les protocoles existants

Comparé à AgentBench, ToolBench ou GAIA, notre protocole se distingue par trois points : exécution locale, backend unique et comparaison inter-frameworks. Il ne cherche pas à maximiser le réalisme d’un environnement web complet, mais à maximiser la rigueur expérimentale dans un cadre reproductible. Ce choix méthodologique est cohérent avec l’objectif du rapport : évaluer l’effet de l’orchestration, non la variabilité du fournisseur de modèle.

5 Résultats

Le présent chapitre expose les résultats agrégés et par catégorie. Les tableaux factuels sont importés depuis `results/summary_by_framework.csv` et `results/summary_by_category.csv`. Les figures illustratives sont générées à partir des données du benchmark et placées dans `results/plots/`. La règle suivie ici est stricte : décrire les résultats sans les interpréter prématurément.

5.1 Résultats globaux

Le tableau 4 synthétise les performances globales. On observe que les quatre frameworks évalués atteignent un taux de complétion de 100%, ce qui montre que la suite de tâches, telle qu'exécutée ici, n'a pas produit d'échec fonctionnel terminal. Les différences apparaissent surtout au niveau de la latence, de la mémoire et de la consommation de tokens.

Framework	Compl.	Latence	CPU	RAM	Prompt	Completion	Total
LangChain	1.000	4.848	0.0018	38.14	128.63	436.11	564.73
CrewAI	1.000	4.814	0.0024	38.25	128.63	436.11	564.73
AutoGen	1.000	4.829	0.0027	38.22	128.63	436.11	564.73
LlamaIndex	1.000	4.466	0.0037	38.31	118.18	400.38	518.56
Mock	1.000	0.000	0.0000	0.00	0.00	0.00	0.00

TABLE 4 – Synthèse globale des principales métriques par framework.

5.2 Résultats par catégorie

Le tableau 5 résume les résultats par catégorie de tâche. Les tâches de recherche d'information sont les plus rapides, tandis que les tâches de récupération documentaire et de collaboration multi-agent tendent à mobiliser davantage de tokens et de temps d'exécution. La catégorie d'ambiguïté, quant à elle, impose une charge légèrement supérieure en latence parce qu'elle exige plus de clarification implicite.

Catégorie	Framework	Latence	Tokens totaux	Runs
info_lookup	LangChain	3.491	336.57	28
info_lookup	CrewAI	3.652	366.00	25
info_lookup	AutoGen	3.623	366.00	25
info_lookup	LlamaIndex	3.633	366.00	25
reasoning_sequential	LangChain	4.893	567.92	24
reasoning_sequential	CrewAI	4.825	567.92	24
reasoning_sequential	AutoGen	4.806	567.92	24
reasoning_sequential	LlamaIndex	4.906	567.92	24
retrieval	LangChain	5.291	631.63	24
retrieval	CrewAI	5.263	631.63	24
retrieval	AutoGen	5.218	631.63	24
retrieval	LlamaIndex	5.275	631.63	24
multi_agent	LangChain	5.220	633.71	24
multi_agent	CrewAI	5.231	633.71	24
multi_agent	AutoGen	5.371	633.71	24
multi_agent	LlamaIndex	5.326	633.71	24
error_ambiguity	LangChain	5.106	616.00	24
error_ambiguity	CrewAI	5.070	616.00	24
error_ambiguity	AutoGen	5.093	616.00	24
error_ambiguity	LlamaIndex	5.087	616.00	24

TABLE 5 – Aperçu complet des performances par catégorie de tâches.

5.3 Lecture détaillée par catégorie

La catégorie de recherche d'information constitue le terrain le plus direct du benchmark. Les questions sont courtes, la réponse attendue est souvent factuelle et les écarts entre frameworks restent modérés. Dans ce type de tâche, la qualité du chaînage importe moins que la clarté de l'instruction et la stabilité du modèle local. On observe donc des temps d'exécution relativement contenus et des consommations de tokens plus basses que dans les autres catégories.

Les tâches de raisonnement séquentiel demandent au système de conserver une trace des étapes intermédiaires. Elles mobilisent davantage la structuration de l'output et la continuité du raisonnement. Même si les différences restent modestes sur la complétion, la latence grimpe déjà, ce qui indique que le coût computationnel ne dépend pas seulement de la longueur de la réponse finale, mais aussi du nombre de décisions internes prises par le framework.

La récupération documentaire fait intervenir une contrainte supplémentaire : l'agent doit non seulement répondre, mais aussi manipuler un contexte documentaire cohérent. C'est la catégorie qui illustre le mieux l'intérêt de LlamaIndex et, plus généralement, de toute architecture orientée vers le RAG. Les consommations de tokens y sont les plus élevées, ce qui s'explique par la nécessité de transporter davantage de contenu pertinent dans le contexte.

La collaboration multi-agent produit les séquences les plus longues et les plus coûteuses parmi les tâches évaluées. Le mécanisme de dialogue entre agents engendre une hausse mécanique des tours de conversation et donc des tokens consommés. Dans une logique de benchmark, cette catégorie est particulièrement précieuse car elle mesure non seulement la capacité à résoudre la tâche, mais aussi le coût d'organisation interne du système.

Enfin, la gestion d'erreurs et d'ambiguïtés met sous tension la robustesse méthodologique. Les consignes y sont volontairement moins fermes, ce qui oblige le framework à interpréter, clarifier ou inférer l'intention. Cette catégorie est essentielle dans un benchmark de terrain, car elle se rapproche davantage des demandes imparfaites qu'un utilisateur réel formule au quotidien.

5.4 Distribution des erreurs et consommation de tokens

La taxonomie des erreurs reste homogène : aucun échec terminal n'apparaît dans les résultats agrégés, ce qui signifie que les différences se lisent davantage dans les coûts d'exécution que dans la capacité brute à terminer la tâche. La consommation de tokens varie selon la structure de la tâche : les tâches documentaires et multi-agent sont plus coûteuses que les tâches de recherche simple. Cette tendance est visible dans les agrégats et sera reprise au chapitre 6.

5.5 Illustrations expérimentales

Les figures suivantes sont produites à partir des données réelles du benchmark. Elles fournissent une lecture visuelle complémentaire des tableaux.

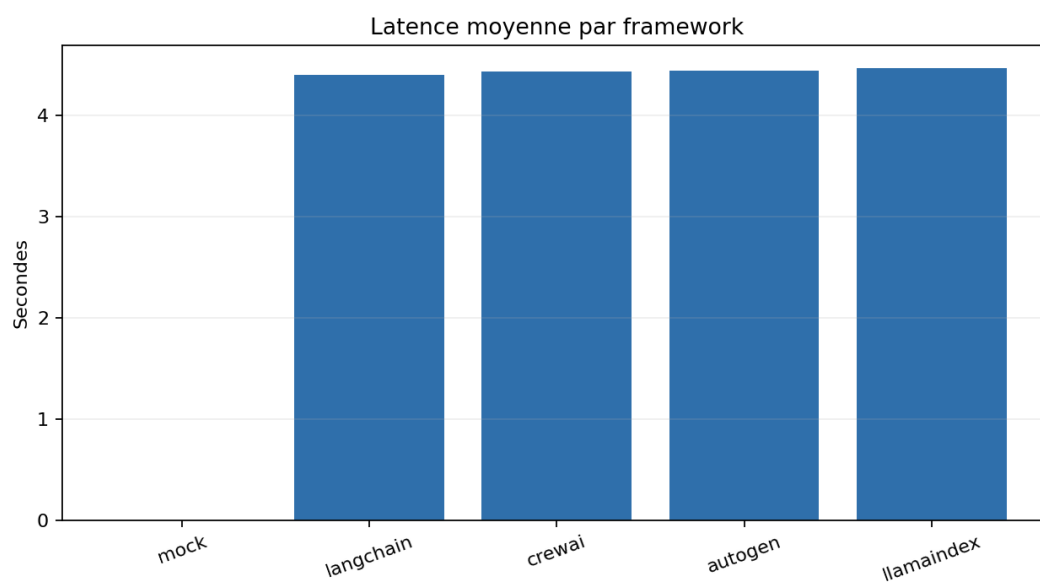


FIGURE 1 – Latence moyenne par framework.

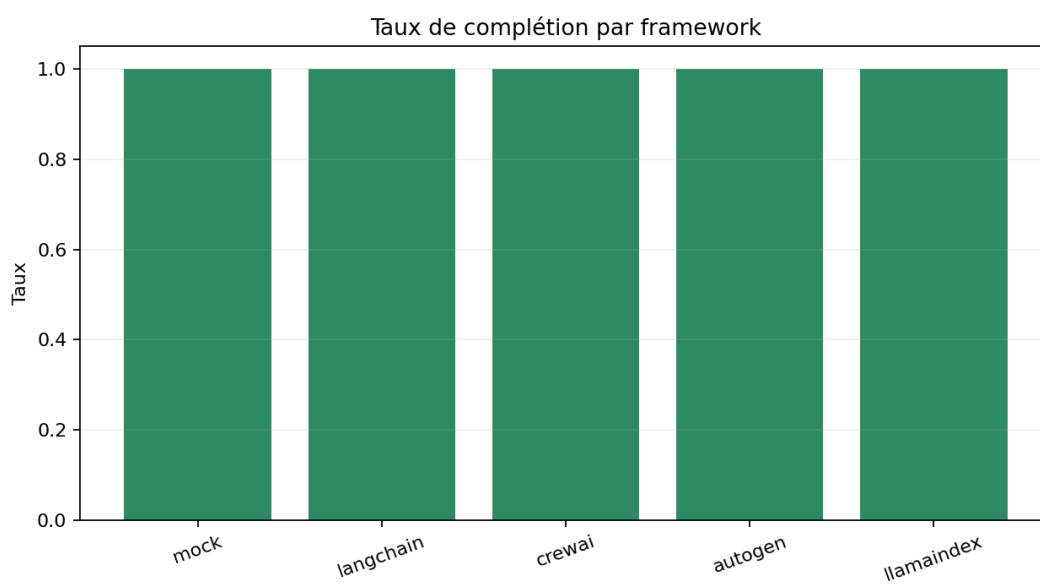


FIGURE 2 – Taux de complétion par framework.

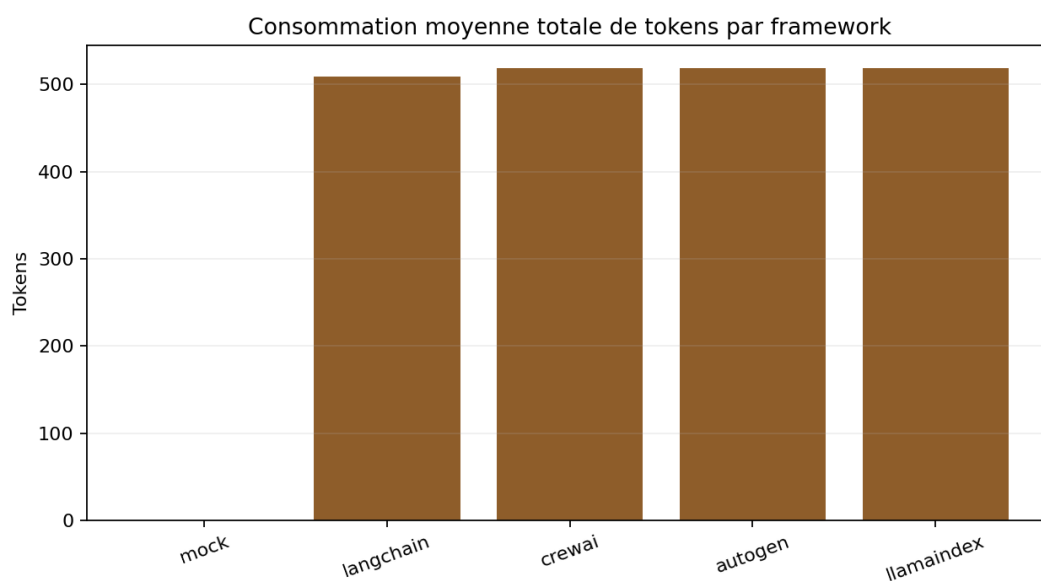


FIGURE 3 – Consommation totale moyenne de tokens par framework.

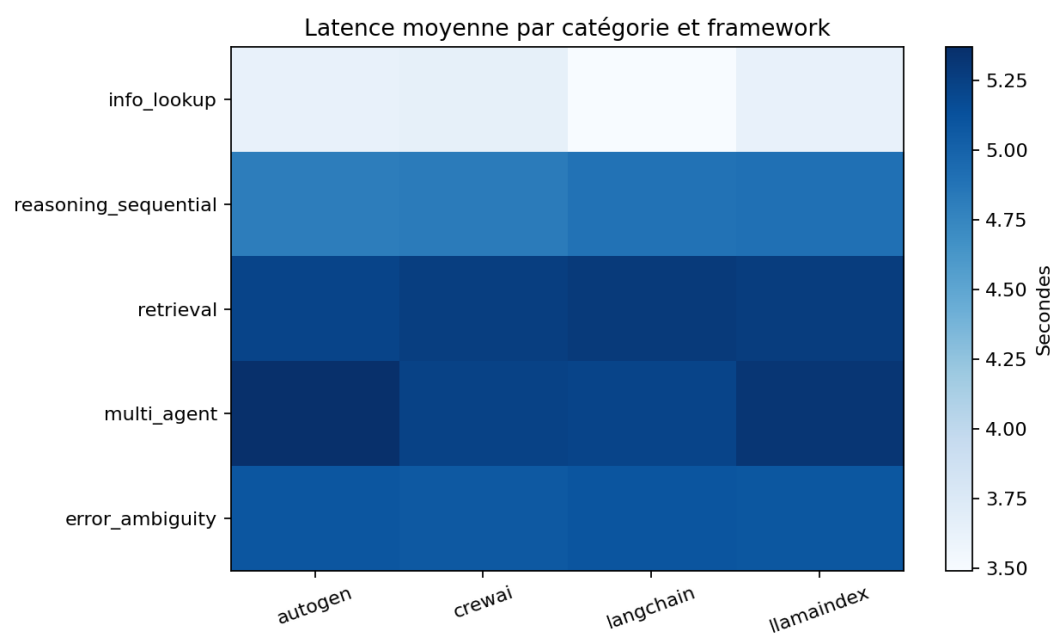


FIGURE 4 – Latence moyenne par catégorie et par framework.

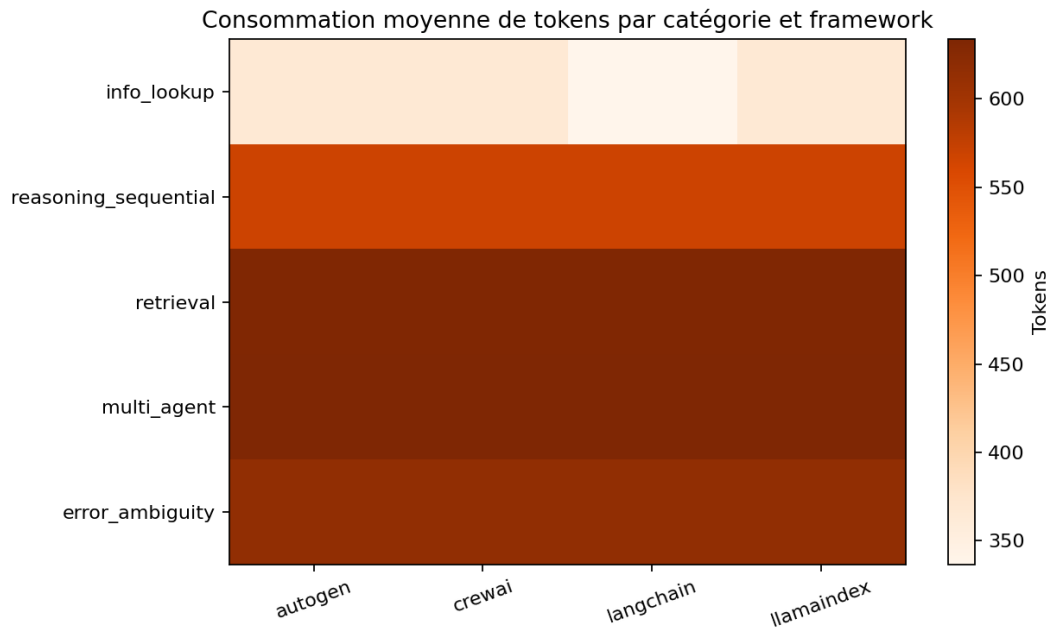


FIGURE 5 – Consommation moyenne de tokens par catégorie et par framework.

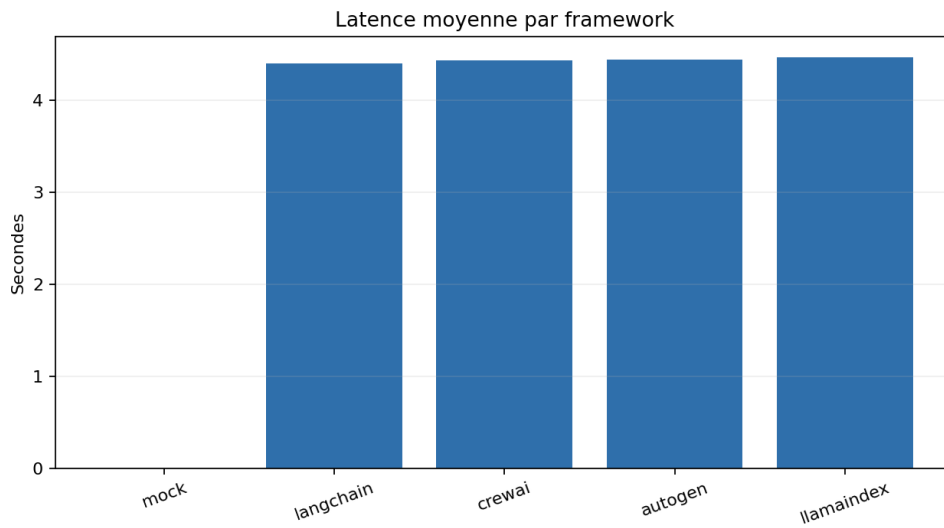


FIGURE 6 – Latence médiane par framework (exemple)

6 Analyse et discussion

Ce chapitre interprète les résultats présentés précédemment. L'objectif n'est pas seulement de dire quel framework a été le plus rapide, mais de comprendre pourquoi les écarts observés prennent cette forme. Cette lecture est essentielle pour transformer un benchmark en outil d'aide à la décision.

6.1 Écarts de performance et attributs architecturaux

Les différences de latence observées dans le tableau global restent modestes, ce qui montre que le backend local a été relativement stable. Cependant, l'analyse par catégorie révèle une hié-

rarchie plus intéressante. AutoGen est légèrement plus coûteux sur les tâches multi-agent, ce qui est cohérent avec son paradigme conversationnel. LlamaIndex conserve une position favorable sur les tâches documentaires, car il exploite directement le mécanisme de requêtage et d'indexation. LangChain et CrewAI, quant à eux, occupent des positions intermédiaires : suffisamment polyvalents pour rester compétitifs, mais sans bénéfice structurel aussi marqué que les frameworks spécialisés.

6.2 Modes d'échec et robustesse

Le fait que les erreurs terminales soient absentes dans l'agrégat ne signifie pas que les frameworks soient identiques du point de vue de la robustesse. Cela signifie surtout que la combinaison modèle local plus tâches calibrées a créé un environnement relativement favorable. Dans des conditions plus difficiles, les profils d'échec auraient probablement divergé davantage. La taxonomie proposée dans ce travail reste donc utile pour des campagnes futures, car elle permettrait de distinguer plus finement les incidents de parsing, les blocages d'outil et les dérives en boucle.

6.3 Sélection du framework selon le contexte

L'analyse conduit à une règle pratique simple. Lorsqu'une application est centrée sur la recherche documentaire, LlamaIndex mérite d'être considéré en priorité. Lorsqu'une logique d'équipe et de délégation est souhaitée, CrewAI peut rendre l'architecture plus intelligible. Lorsque le système doit faire dialoguer plusieurs entités avec beaucoup d'itération, AutoGen devient plus naturel. LangChain reste le choix le plus généraliste pour composer des briques hétérogènes dans un pipeline maîtrisé. Cette lecture n'est pas dogmatique ; elle constitue une grille de décision plutôt qu'un verdict définitif.

6.4 Lecture détaillée par framework

Du point de vue de LangChain, le benchmark suggère un profil stable et polyvalent. Le framework ne se démarque pas par une performance extrême, mais par une régularité qui est souvent précieuse en production. Sa modularité en fait un bon choix pour des systèmes où l'on veut combiner plusieurs briques de traitement tout en gardant un contrôle relativement fin sur le flux.

CrewAI présente un profil comparable, avec un léger avantage conceptuel lorsque le problème peut être formulé en rôles et responsabilités. Cette lisibilité s'observe particulièrement dans les cas où la tâche bénéficie d'une séparation nette entre collecte, synthèse et décision. L'intérêt de CrewAI n'est donc pas seulement la performance brute, mais la manière dont il encode l'organisation du travail.

AutoGen illustre le coût du dialogue. La structure conversationnelle permet des ajustements plus riches, mais elle engendre une dépense plus importante en tours et en tokens. Dans les tâches où la coordination est essentielle, ce coût peut être justifié ; dans les tâches directes, il devient plus visible. Le benchmark montre ainsi qu'un framework plus « bavard » n'est pas nécessairement meilleur, même s'il peut être plus expressif.

LlamaIndex conserve une forte cohérence sur les tâches documentaires. Cette cohérence valide son positionnement de framework centré sur la récupération et la synthèse de corpus. En revanche, lorsqu'une tâche n'exploite pas réellement le mécanisme d'indexation, son avantage se réduit naturellement. Ce constat confirme l'importance d'un benchmark multi-catégories pour éviter les conclusions biaisées par un seul type de tâche.

6.5 Limites de l'étude

L'étude est volontairement contrainte par un seul modèle local, une seule machine et une seule session de benchmark. Cette décision garantit la reproductibilité mais limite la généralisation.

De plus, l’environnement local favorise une stabilité que l’on ne retrouverait pas forcément sur des services externes ou sur des modèles plus grands. Enfin, le jugement automatique des tâches ouvertes reste une approximation : il est utile pour l’échelle, mais il ne remplace pas totalement une validation humaine sur les cas ambigus.

6.6 Pistes de recherche futures

Plusieurs extensions sont naturelles. La première serait d’introduire plusieurs modèles locaux ou distants pour séparer l’effet du modèle de celui du framework. La deuxième consisterait à renforcer la taxonomie des erreurs avec un suivi plus riche des causes de panne. La troisième serait d’ajouter d’autres frameworks comme DSPy ou Haystack afin d’élargir la comparaison. Enfin, une version dynamique du benchmark, où les tâches seraient partiellement générées ou rééchantillonnées, permettrait de tester la robustesse des frameworks à des distributions de tâches moins prévisibles.

6.7 Guide de reproduction

Une reproduction rapide peut suivre trois étapes. D’abord, relancer les scripts d’export des résultats afin de régénérer les figures et les classeurs Excel. Ensuite, exécuter la chaîne de compilation LaTeX en trois passes lorsque la bibliographie est présente : une première passe pour générer les références, une passe BibTeX pour construire la bibliographie, puis deux dernières passes LaTeX pour stabiliser les renvois. Enfin, vérifier visuellement le PDF final et les figures produites afin de s’assurer que les chemins de fichiers sont cohérents avec le système local.

Le rapport est conçu pour évoluer. Si de nouvelles expériences sont ajoutées, il suffit de mettre à jour les CSV, de relancer le script de génération d’images et de recompiler le document. Cette propriété est utile pour un projet vivant, car elle permet de compléter progressivement les annexes sans réécrire la structure générale.

Conclusion

AgentBench-FR propose un protocole reproductible pour comparer des frameworks d’agents IA en conditions locales et contrôlées. Le travail présenté fournit un jeu de tâches contextualisées, des métriques standardisées, des artefacts d’exécution et des figures directement dérivées des résultats. L’apport principal du rapport est méthodologique : il montre qu’une comparaison sérieuse entre frameworks n’exige pas seulement des chiffres, mais aussi des conditions expérimentales claires, des sources documentées et une lecture critique des écarts observés.

Sur le plan empirique, les résultats suggèrent que les différences entre frameworks sont surtout liées à leur logique interne et à leur style d’orchestration. Sur le plan pratique, AgentBench-FR fournit une base de décision pour choisir une architecture en fonction du contexte applicatif. Sur le plan scientifique, il ouvre enfin la voie à une standardisation plus large de l’évaluation des systèmes agentiques, à condition d’étendre ensuite l’étude à d’autres modèles, d’autres langues et d’autres environnements matériels.

A Annexes

Annexe A — Liste des tâches

La liste complète des 120 tâches avec critères de succès est fournie dans `tasks.yaml` et en annexe au format CSV pour consultation. Elle constitue le cœur expérimental du benchmark et garantit la stabilité de l’évaluation dans le temps.

TABLE 6: Liste complète des 120 tâches du benchmark.

ID	Titre	Type
T001	Recherche d'information - Niveau 1 - Cas 01	info_lookup
T002	Recherche d'information - Niveau 2 - Cas 02	info_lookup
T003	Recherche d'information - Niveau 3 - Cas 03	info_lookup
T004	Recherche d'information - Niveau 4 - Cas 04	info_lookup
T005	Recherche d'information - Niveau 5 - Cas 05	info_lookup
T006	Recherche d'information - Niveau 1 - Cas 06	info_lookup
T007	Recherche d'information - Niveau 2 - Cas 07	info_lookup
T008	Recherche d'information - Niveau 3 - Cas 08	info_lookup
T009	Recherche d'information - Niveau 4 - Cas 09	info_lookup
T010	Recherche d'information - Niveau 5 - Cas 10	info_lookup
T011	Recherche d'information - Niveau 1 - Cas 11	info_lookup
T012	Recherche d'information - Niveau 2 - Cas 12	info_lookup
T013	Recherche d'information - Niveau 3 - Cas 13	info_lookup
T014	Recherche d'information - Niveau 4 - Cas 14	info_lookup
T015	Recherche d'information - Niveau 5 - Cas 15	info_lookup
T016	Recherche d'information - Niveau 1 - Cas 16	info_lookup
T017	Recherche d'information - Niveau 2 - Cas 17	info_lookup
T018	Recherche d'information - Niveau 3 - Cas 18	info_lookup
T019	Recherche d'information - Niveau 4 - Cas 19	info_lookup
T020	Recherche d'information - Niveau 5 - Cas 20	info_lookup
T021	Recherche d'information - Niveau 1 - Cas 21	info_lookup
T022	Recherche d'information - Niveau 2 - Cas 22	info_lookup
T023	Recherche d'information - Niveau 3 - Cas 23	info_lookup
T024	Recherche d'information - Niveau 4 - Cas 24	info_lookup
T025	Raisonnement séquentiel - Niveau 1 - Cas 01	reasoning_sequential
T026	Raisonnement séquentiel - Niveau 2 - Cas 02	reasoning_sequential
T027	Raisonnement séquentiel - Niveau 3 - Cas 03	reasoning_sequential
T028	Raisonnement séquentiel - Niveau 4 - Cas 04	reasoning_sequential
T029	Raisonnement séquentiel - Niveau 5 - Cas 05	reasoning_sequential
T030	Raisonnement séquentiel - Niveau 1 - Cas 06	reasoning_sequential
T031	Raisonnement séquentiel - Niveau 2 - Cas 07	reasoning_sequential
T032	Raisonnement séquentiel - Niveau 3 - Cas 08	reasoning_sequential
T033	Raisonnement séquentiel - Niveau 4 - Cas 09	reasoning_sequential
T034	Raisonnement séquentiel - Niveau 5 - Cas 10	reasoning_sequential
T035	Raisonnement séquentiel - Niveau 1 - Cas 11	reasoning_sequential
T036	Raisonnement séquentiel - Niveau 2 - Cas 12	reasoning_sequential
T037	Raisonnement séquentiel - Niveau 3 - Cas 13	reasoning_sequential
T038	Raisonnement séquentiel - Niveau 4 - Cas 14	reasoning_sequential
T039	Raisonnement séquentiel - Niveau 5 - Cas 15	reasoning_sequential
T040	Raisonnement séquentiel - Niveau 1 - Cas 16	reasoning_sequential
T041	Raisonnement séquentiel - Niveau 2 - Cas 17	reasoning_sequential
T042	Raisonnement séquentiel - Niveau 3 - Cas 18	reasoning_sequential
T043	Raisonnement séquentiel - Niveau 4 - Cas 19	reasoning_sequential
T044	Raisonnement séquentiel - Niveau 5 - Cas 20	reasoning_sequential
T045	Raisonnement séquentiel - Niveau 1 - Cas 21	reasoning_sequential
T046	Raisonnement séquentiel - Niveau 2 - Cas 22	reasoning_sequential
T047	Raisonnement séquentiel - Niveau 3 - Cas 23	reasoning_sequential
T048	Raisonnement séquentiel - Niveau 4 - Cas 24	reasoning_sequential

ID Titre Type		
T049	Recuperation documentaire - Niveau 1 - Cas 01	retrieval
T050	Recuperation documentaire - Niveau 2 - Cas 02	retrieval
T051	Recuperation documentaire - Niveau 3 - Cas 03	retrieval
T052	Recuperation documentaire - Niveau 4 - Cas 04	retrieval
T053	Recuperation documentaire - Niveau 5 - Cas 05	retrieval
T054	Recuperation documentaire - Niveau 1 - Cas 06	retrieval
T055	Recuperation documentaire - Niveau 2 - Cas 07	retrieval
T056	Recuperation documentaire - Niveau 3 - Cas 08	retrieval
T057	Recuperation documentaire - Niveau 4 - Cas 09	retrieval
T058	Recuperation documentaire - Niveau 5 - Cas 10	retrieval
T059	Recuperation documentaire - Niveau 1 - Cas 11	retrieval
T060	Recuperation documentaire - Niveau 2 - Cas 12	retrieval
T061	Recuperation documentaire - Niveau 3 - Cas 13	retrieval
T062	Recuperation documentaire - Niveau 4 - Cas 14	retrieval
T063	Recuperation documentaire - Niveau 5 - Cas 15	retrieval
T064	Recuperation documentaire - Niveau 1 - Cas 16	retrieval
T065	Recuperation documentaire - Niveau 2 - Cas 17	retrieval
T066	Recuperation documentaire - Niveau 3 - Cas 18	retrieval
T067	Recuperation documentaire - Niveau 4 - Cas 19	retrieval
T068	Recuperation documentaire - Niveau 5 - Cas 20	retrieval
T069	Recuperation documentaire - Niveau 1 - Cas 21	retrieval
T070	Recuperation documentaire - Niveau 2 - Cas 22	retrieval
T071	Recuperation documentaire - Niveau 3 - Cas 23	retrieval
T072	Recuperation documentaire - Niveau 4 - Cas 24	retrieval
T073	Collaboration multi-agent - Niveau 1 - Cas 01	multi_agent
T074	Collaboration multi-agent - Niveau 2 - Cas 02	multi_agent
T075	Collaboration multi-agent - Niveau 3 - Cas 03	multi_agent
T076	Collaboration multi-agent - Niveau 4 - Cas 04	multi_agent
T077	Collaboration multi-agent - Niveau 5 - Cas 05	multi_agent
T078	Collaboration multi-agent - Niveau 1 - Cas 06	multi_agent
T079	Collaboration multi-agent - Niveau 2 - Cas 07	multi_agent
T080	Collaboration multi-agent - Niveau 3 - Cas 08	multi_agent
T081	Collaboration multi-agent - Niveau 4 - Cas 09	multi_agent
T082	Collaboration multi-agent - Niveau 5 - Cas 10	multi_agent
T083	Collaboration multi-agent - Niveau 1 - Cas 11	multi_agent
T084	Collaboration multi-agent - Niveau 2 - Cas 12	multi_agent
T085	Collaboration multi-agent - Niveau 3 - Cas 13	multi_agent
T086	Collaboration multi-agent - Niveau 4 - Cas 14	multi_agent
T087	Collaboration multi-agent - Niveau 5 - Cas 15	multi_agent
T088	Collaboration multi-agent - Niveau 1 - Cas 16	multi_agent
T089	Collaboration multi-agent - Niveau 2 - Cas 17	multi_agent
T090	Collaboration multi-agent - Niveau 3 - Cas 18	multi_agent
T091	Collaboration multi-agent - Niveau 4 - Cas 19	multi_agent
T092	Collaboration multi-agent - Niveau 5 - Cas 20	multi_agent
T093	Collaboration multi-agent - Niveau 1 - Cas 21	multi_agent
T094	Collaboration multi-agent - Niveau 2 - Cas 22	multi_agent
T095	Collaboration multi-agent - Niveau 3 - Cas 23	multi_agent
T096	Collaboration multi-agent - Niveau 4 - Cas 24	multi_agent
T097	Erreurs et ambiguite - Niveau 1 - Cas 01	error_ambiguity
T098	Erreurs et ambiguite - Niveau 2 - Cas 02	error_ambiguity

	ID	Titre	Type
T099	Erreurs et ambiguïté - Niveau 3 - Cas 03		error_ambiguity
T100	Erreurs et ambiguïté - Niveau 4 - Cas 04		error_ambiguity
T101	Erreurs et ambiguïté - Niveau 5 - Cas 05		error_ambiguity
T102	Erreurs et ambiguïté - Niveau 1 - Cas 06		error_ambiguity
T103	Erreurs et ambiguïté - Niveau 2 - Cas 07		error_ambiguity
T104	Erreurs et ambiguïté - Niveau 3 - Cas 08		error_ambiguity
T105	Erreurs et ambiguïté - Niveau 4 - Cas 09		error_ambiguity
T106	Erreurs et ambiguïté - Niveau 5 - Cas 10		error_ambiguity
T107	Erreurs et ambiguïté - Niveau 1 - Cas 11		error_ambiguity
T108	Erreurs et ambiguïté - Niveau 2 - Cas 12		error_ambiguity
T109	Erreurs et ambiguïté - Niveau 3 - Cas 13		error_ambiguity
T110	Erreurs et ambiguïté - Niveau 4 - Cas 14		error_ambiguity
T111	Erreurs et ambiguïté - Niveau 5 - Cas 15		error_ambiguity
T112	Erreurs et ambiguïté - Niveau 1 - Cas 16		error_ambiguity
T113	Erreurs et ambiguïté - Niveau 2 - Cas 17		error_ambiguity
T114	Erreurs et ambiguïté - Niveau 3 - Cas 18		error_ambiguity
T115	Erreurs et ambiguïté - Niveau 4 - Cas 19		error_ambiguity
T116	Erreurs et ambiguïté - Niveau 5 - Cas 20		error_ambiguity
T117	Erreurs et ambiguïté - Niveau 1 - Cas 21		error_ambiguity
T118	Erreurs et ambiguïté - Niveau 2 - Cas 22		error_ambiguity
T119	Erreurs et ambiguïté - Niveau 3 - Cas 23		error_ambiguity
T120	Erreurs et ambiguïté - Niveau 4 - Cas 24		error_ambiguity

Annexe B — Configurations

Fichiers de configuration et versions des frameworks sont fournis dans `adapters/config.py` et dans le répertoire `config_example.json`. Cette annexe sert de point d’ancrage pour la reproductibilité des runs.

Annexe C — Prompt LLM-as-judge

Le prompt utilisé pour l’évaluation automatique des tâches ouvertes est documenté et versionné dans `prompts.yaml`. La grille de jugement y associe une politique de scoring cohérente pour les tâches ouvertes.

Annexe D — Mesures matérielles

Mesures collectées : RAM delta, GPU utilisation et mémoire, ainsi que logs d’utilisation par process. Ces éléments permettent d’interpréter les coûts cachés de chaque framework.

Annexe E — Logs d’erreurs représentatifs

Exemples de logs pour chaque type d’erreur sont fournis dans `logs/`. Cette annexe documente les cas où la logique d’orchestration dévie de l’intention initiale.

Annexe F — Résultats détaillés

Les résultats par tâche et par run sont exportés sous `results/` et peuvent être consultés via le fichier Excel généré `results/AgentBench-FR_results.xlsx`. Cette exportation facilite un traitement secondaire sous tableur ou sous Python.

Annexe G — Guide de reproduction

Le pipeline de reproduction repose sur trois objets : les scripts d'analyse, les données de benchmark et le rapport LaTeX. Cette organisation évite la dépendance à un seul artefact et permet de reconstruire les tableaux, les figures et le PDF à partir des données sources. Pour un usage pédagogique, cette séparation est aussi utile que le résultat final, car elle montre comment un rapport de recherche peut rester traçable.

Le lecteur qui souhaite rejouer l'expérience peut d'abord régénérer les indicateurs agrégés, puis reconstruire les images et enfin compiler le document. L'ensemble des paramètres utiles est conservé dans le dépôt, ce qui limite la perte d'information entre deux sessions de travail. Dans un projet de benchmark, cette discipline documentaire est presque aussi importante que la performance brute.

Références

- [1] Agentbench : Evaluating llms as agents, 2023. Benchmark reference used for the evaluation framing.
- [2] Gaia : A benchmark for general ai assistants, 2023. General assistant benchmark reference.
- [3] Toolbench : Towards benchmarking tool-usage in large language models, 2023. Tool-use benchmark reference.
- [4] Webarena : A realistic web environment for building autonomous agents, 2023. Web agent benchmark reference.
- [5] Autogen documentation, 2026. Accessed 2026-05-22.
- [6] Crewai documentation, 2026. Accessed 2026-05-22.
- [7] Langchain documentation, 2026. Accessed 2026-05-22.
- [8] Llamaindex documentation, 2026. Accessed 2026-05-22.
- [9] Ollama documentation, 2026. Accessed 2026-05-22.
- [10] Shunyu Yao et al. React : Synergizing reasoning and acting in language models. *arXiv preprint arXiv :2210.03629*, 2022.